FORENSIC INCIDENT REPORT & POST-MORTEM

CLASSIFICATION: PUBLIC / TLP:CLEAR

71 Days in the Dark: How an 'Innocent' Browser Extension Hooked My Digital Life (and the Antivirus Lie)

AUTHOR & LEAD INVESTIGATOR
Christos Kataxenos (christoskataxenos.com)

INCIDENT CLOSE DATE
2026-10-31

SYSTEM / ARCHITECTURE
Chromium / Windows 11 / Debian / Android

Executive Summary: *Anatomy of a real browser hijacking incident. 71 days of stealth operation, 1,835 Malwarebytes blocks providing false confidence, silent DOM injections, and how my Discord account was turned into a stealth MrBeast scam bot.*

`#cybersecurity` `#incident-response` `#malware-analysis` `#browser-security` `#discord` `#forensics` `#windows` `#programming-humor`

```
<div class="stat-header">Panic Level</div>
<div class="stat-num">95%</div>
<div class="stat-bar"><div class="stat-fill" style="width: 95%"></div></div>
```

```
<div class="stat-header">Freddo Espressos</div>
<div class="stat-num">18 <span class="stat-unit">cups</span></div>
<div class="stat-sub">High-octane fuel</div>
```

```
<span class="callout-icon">🚨</span>
<span class="callout-title">The Hard Lesson</span>
```

```
<p>Just because your antivirus throws a “Malicious connection blocked”
```

alert does not mean your workstation is clean. It only means it blocked what was already on its blacklist. Domains missing from that list continue talking directly to your browser DOM over HTTP/3 QUIC with 20ms latency.

As a computer science student at Hellenic Open University (EAP) and a developer constantly working with code, pointers, and system logs, I held that familiar belief: *“I know what I’m doing. I don’t run random executables, I have a network-wide DNS sinkhole, and real-time security is enabled. I’m fine.”*

On October 31, 2026, reality showed me otherwise.

Digging through my workstation’s local storage and network databases, I uncovered that for **71 days** (since August 21), my Chromium browser on my primary desktop PC (specifically Microsoft Edge, though the architecture affects all Chromium-based browsers — I verified this across my Debian Linux partition and Linux laptop) had been silently executing third-party scripts inside the DOM. It hooked into every tab I opened: GitHub, AWS, Microsoft accounts, online banking, e-commerce stores, and my university portal.

The frustrating part? It did all this while my endpoint antivirus reported green checkmarks every single day.

---

1. Infection Timeline: August 21, 2026

On August 21, I downloaded a software utility from `filecr.com`. As it turned out, that download silently side-loaded the **WebCRX** (`filecr-assistant`) extension into Microsoft Edge without prompting for permissions.

Examining Edge’s `Preferences` file (under `updateclientdata`), the forensic trail was undeniable: * **Extension ID:** `ddgilliopjknmglnpkegbjpoilgachlm` (`WebCRX` / `filecr-assistant`) * **Installation Date:** August 21, 2026 * **Discovery & Removal Date:** October 31, 2026 * **Active Dwell Time: 71 days** (verified by the delta between the browser’s internal `dlrc` and `installdate` timestamps: exactly 71 days)

The exact same day, Hola ad remover (`lalfpjdbhpmnhfofkckdpkljeilmogfl`) appeared in the profile, followed two days later by CocoCut (`npdhgjfiikhgncaacnnodpfchmelnchm`).

Here is the first anomaly: **I never intentionally installed these extra extensions.** When you run Pi-hole and AdGuard Home on your local network to block ads and trackers at the DNS level, installing an ad-remover extension makes zero sense. These extensions were dropped via drive-by bundling (side-loading) or pushed in the background without user confirmation.

The extension settled quietly into the `Default` profile. It showed no popups, caused no CPU spikes, and gave no visual indications of infection. It simply created a recurring background alarm named `webcrx_auto_update`, which fired **every 30 minutes**.

---

2. The Antivirus Illusion: 1,835 Blocks Masking the Truth

On October 23, Malwarebytes began firing desktop notifications: > *«Malicious connection blocked: msedge.exe -> overbridgenet.com (TCP:443)»*

Inside the log directory (`C:\ProgramData\Malwarebytes\MBAMService\MwacDetections`), I counted **1,835 blocked outbound connections** to `overbridgenet.com` (`/jsv8/offer`), alongside secondary requests from `smartscreen.exe`.

My immediate reaction was relief: *“Good, the antivirus is doing its job. It stopped 1,835 connections, the machine is safe.”*

That assumption was wrong.

Inspecting Edge's network database (`Network Persistent State`), I verified that the SRTT (Smoothed Round-Trip Time) for `overbridgenet.com` was `None` . The antivirus was indeed dropping 100% of packets destined for *that specific server*.

However, the threat actors operated a distributed Command and Control (C2) network. The antivirus knew about only one node.

3. Four Unblocked C2 Endpoints Operating in Plain Sight

While the antivirus blocked the burned domain, four other endpoints in the same infrastructure communicated freely with my extension over **HTTP/3 QUIC**, enjoying fast response times (19ms – 29ms):

Destination Server	Traffic Source	Status	Function
`https://rapi.acrop.net/blocklist.json`	`chrome-extension://ddgilliop...`	**SUCCESS** (SRTT: 20.9 ms)	**Primary C2:** Delivered dynamic configs every 30 minutes via the `webcrx_auto_update` alarm.
`https://api.jetpusher.com`	`chrome-extension://ddgilliop...`	**SUCCESS** (SRTT: 23.2 ms)	**Telemetry Ingest:** The endpoint where the extension reported browser activity.
`https://analytics-api.zintrack.com` & `dash.zintrack.com`	`chrome-extension://ddgilliop...`	**SUCCESS** (SRTT: 22.9 ms)	**Tracking Beacon & Push Scripts:** Served external payloads (`/js/push.js?p=2&v=1.0.2`).
`https://static-lib.com`	**Within EVERY Web Page!**	**SUCCESS** (SRTT: 19–29 ms)	**Silent Script CDN (SCDN):** Injected 3 external JavaScript files directly into the DOM of every tab!

```
<span class="callout-icon">⚠</span>
<span class="callout-title">Why Didn't Endpoint Protections Intervene?</span>
```

```
<p>All traffic originated from Microsoft's signed binary
```

(`msedge.exe`) using standard HTTPS/QUIC ports (443). To the host OS and standard firewalls, this looked like normal web browsing.

4. Silent DOM Injections: What Was Running in the Background

Examining the cache directory: `C:\Users\chris\AppData\Local\Microsoft\Edge\User Data\Default\Code Cache\js` showed the actual scope of activity.

Over just **six days** (October 26 – October 31), the cache recorded **724 executions** of three external script modules injected into open tabs: 1. `https://static-lib.com/s/6be...vendor.min.js?v=61.9...&r=DE&t=<JWT>` (**327 runs**) 2. `https://static-lib.com/l/3af...module.core.js?lv=13&t=<JWT>` (**199 runs**) 3. `https://static-lib.com/s/10f...base.min.js?v=27.9...&r=ZZ&t=<JWT>` (**198 runs**)

What were these scripts doing?

Decoding the Base64 JWT tokens (`&t=eyJ...`) and reading the LevelDB store (`Local Storage\leveldb\005183.ldb` , `005187.ldb`):

- **Full Origin & Referer URLs:** The scripts recorded the exact URL of each active page.
- **Signed Session JWTs:** Session metadata attached to network calls (`{"h":"static-lib.com", "jti":"...", "iat":1793029228, ...}`).
- **Persistent LocalStorage Tracking IDs:** The script wrote `__scdn_ga_sid` and `__3g4_session_id` directly into the `localStorage` of visited origins (e.g. writing `{"id":"jzxec9oyn9jn58m6j8bv1"}` on `login.live.com`), maintaining tracking even across IP changes.
- **Direct DOM Access:** Running in-page meant full access to whatever was rendered: input fields, account details, and **OAuth session tokens** stored in `localStorage` .

5. Threat Model: Services Within Scope

When an external script executes within the page DOM, it makes no distinction between simple blogs and critical services. It executes on whatever tab the user opens.

Inspecting cached code and local storage records showed injections across several key categories:

- **Identity Providers & Single Sign-On:** Primary email portals, cloud infrastructure consoles, and code repositories. Stolen session tokens or OAuth tokens from `localStorage` can allow account access without re-prompting for 2FA.
- **Academic Portals:** University platforms and grading systems.
- **E-Commerce:** Online storefronts with stored payment details, shipping addresses, and transaction history.
- **Cloud & Developer Platforms:** Server dashboards, package registries, and API management consoles.
- **Professional Networks:** Collaboration platforms and messaging tools.

```
<span class="callout-icon">ℹ️</span>
<span class="callout-title">The Reality of Browser Injections</span>
```

```
<p>The script ran even when visiting credential-leak check sites (like
```

Have I Been Pwned) or the documentation portals of the **Antivirus** itself. While checking whether old credentials were safe, the injected script was actively executing inside the page.

---

6. Active Weaponization: My Discord Account Becomes a Stealth MrBeast Scam Bot

Up to this point, one might attempt to rationalize the situation: *"Maybe it was just a passive analytics framework or an affiliate tracker."*

The reality was far more aggressive.

During those same days, across several Discord servers I participate in, the notorious **MrBeast Scam** campaign began circulating — a widespread phishing scam promoting fake $2,500 giveaways and free crypto to lure unsuspecting users into credential-harvesting pages.

The real shock landed when I discovered that **my own Discord account was actively sending out these scam messages to server members and contacts without my knowledge**.

Because the in-process script had access to the browser DOM and local storage session tokens, the attackers hijacked my authenticated Discord session. They turned my legitimate account into an automated spam puppet.

The most insidious element was their automated stealth suppression tactic: 1. The script blasted the scam payload directly to mutual server members and friends. 2. **Immediately after sending the message, it automatically added the recipient to my Discord Ignore / Block list!**

The logic behind this was deviously brilliant: If a friend replied with *"Hey Chris, did you get hacked?"* or *"What kind of link is this?"*, their response never reached me. No pop-up notifications appeared, and no unread badges flashed on my screen. By instantly muting each recipient, the script silenced its victims in real time, preventing me from catching on to the fact that my account was actively weaponized.

That discovery was what sent the incident response into overdrive. This was no longer passive telemetry; it was active identity hijacking.

---

7. Incident Triage & Remote Work: The Citrix NetScaler Scenario

While investigating this incident, another concern surfaced: my wife works remotely using Citrix Workspace, and her employer suspended all remote access due to zero-day exploits targeting **Citrix NetScaler** appliances globally.

During initial triage, an obvious question arose: *"Her corporate laptop shares our home Wi-Fi with my compromised PC. Could lateral movement have occurred across the local network?"*

A technical breakdown clarified the boundaries:

1. **Unrelated Incidents (Correlation ≠ Causation):** The Citrix NetScaler attacks were server-side zero-days targeting enterprise gateways to leak session tokens from memory. They had no connection to local network worms. My workstation's extension had been present since August 21.
2. **Hardware Separation:** My wife works **strictly on her dedicated corporate laptop** and never used my workstation. Because the infection was isolated to the browser process (`msedge.exe`) and did not establish LAN-level propagation, corporate hardware remained completely unaffected.
3. **The Rule for Home Offices:** If she had used my PC under a Bring Your Own Device (BYOD) arrangement, the injected scripts would have had direct access to active session cookies. Strict hardware separation (and placing work devices on an isolated Guest VLAN) prevented cross-contamination.

---

8. Why Local Network DNS Sinkholes Failed to Block C2 Traffic

My home network does not rely on default ISP DNS settings. I run local **DNS sinkholes (Pi-hole / AdGuard Home)**, redundant upstream resolvers, and managed network equipment.

When endpoint monitoring revealed hundreds of outbound connections to malicious endpoints, the key question was: *"Why didn't the local DNS sinkholes block these domains?"*

The post-mortem revealed five distinct issues:

```
[ Microsoft Edge (msedge.exe) ]
       |
       ├──→ 1. DoH Bypass (Secure DNS inside Browser) ──────────────────→ Internet (Local DNS Bypassed)
       |
       ├──→ 2. IPv6 Leak (ISP DNS via SLAAC) ──────────────────────────→ Internet (IPv4 DNS Bypassed)
       |
       └──→ 3. Local DNS Sinkhole Query
                    |
                    ├──→ Domain NOT present in standard adlists
                    |
                    └──→ Upstream Race: Unfiltered resolver answered faster
                         than Security Resolver ──────────────────────────→ Allowed
```

1. Threat Intelligence Coverage (Ad-blocking vs Malware)

Standard ad-blocking lists focus on banners and trackers. C2 servers, fast-flux endpoints, and stealer infrastructure are not categorized as ads. Without specialized botnet and threat intelligence feeds, the sinkhole allows the queries through.

2. The Upstream DNS Race Condition

I normally configure **Quad9** as my primary upstream resolver for its built-in malware blocking. However, audit records revealed that additional public resolvers had been enabled alongside it (likely left over from a previous configuration or update).

This created an upstream race: * The sinkhole dispatched queries to all configured upstream servers simultaneously. * Whichever resolver returned a response first won the race and was served to the client. * Quad9 blocked the domain, but the unfiltered resolver returned the malicious IP. When the unfiltered resolver was faster, the protection was bypassed entirely.

3. IPv6 DNS Leaks

The router advertised IPv6 DNS servers via SLAAC. Windows prioritizes IPv6 by default, allowing many DNS queries to bypass the local IPv4 sinkhole.

4. Browser DoH (Secure DNS) Bypassing Local Policy

Modern browsers promote **DNS-over-HTTPS (DoH)** as a privacy feature.

Without centralized enterprise management: 1. The browser ignores system-level and network-level DNS settings. 2. It sends DNS queries directly over port 443 to public DoH providers. 3. This invalidates local Pi-hole and AdGuard installations, allowing browser extensions and scripts to communicate externally without inspection.

If you run local DNS security: **Disable "Use Secure DNS" in browser settings** or point it explicitly to your local resolver's DoH endpoint.

5. Layer 3 (IP) vs Layer 7 (DNS)

Even when a domain is blocked at the DNS layer, if the browser has a cached IP (`ipconfig /displaydns`) or the script calls an IP address directly, DNS blocking is ineffective. Effective mitigation requires **Layer 3/4 firewall rules** at the host level.

Network Architecture Updates

To close these gaps: 1. **Threat Feeds:** Added dedicated C2, botnet, and anti-malware intelligence lists to the sinkhole. 2. **Upstream Sanitization:** Removed all unfiltered public resolvers, leaving only security-focused resolvers with active DNSSEC and malware filtering. 3. **Failover Redundancy:** Deployed a secondary DNS sinkhole for high availability. 4. **Host Firewalling:** Blocked identified C2 IP addresses directly in Windows Defender Firewall.

Security Control	Before Incident	After Hardening
DNS List Coverage	General ads & trackers only	Threat Intelligence, C2 Botnets & Anti-Malware
C2 Domains	❌ Allowed at DNS layer	✅ Wildcard DNS Block
Upstream Resolution	❌ Race condition with unfiltered resolvers	✅ Security Resolvers only (DNSSEC + Malware Filtering)
DNS Availability	❌ Single point of failure	✅ Redundant failover pair
IP Protection (Layer 3)	⚠️ No blocking on direct IP requests	✅ Host-level Firewall block rules
Network Strategy	⚠️ Passive ad-blocking	✅ Multi-layered active defense

---

9. Operating System Forensics

Before choosing to reformat, I conducted a systematic forensic inspection across the OS:

```
<div class="terminal-dots">
  <span class="t-dot red"></span>
  <span class="t-dot yellow"></span>
  <span class="t-dot green"></span>
</div>
<div class="terminal-title">PowerShell Forensic Investigation</div>
```

```
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-PowerShell/Operational'; Id=4104} |
    Select-Object -First 500 |
    Where-Object { $_.Message -match 'DownloadString|WebClient|IEX|Invoke-RestMethod' }
# Result: 0 suspicious outbound network calls detected.

Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Bits-Client/Operational'; Id=3,4,59,60}
# Result: Only legitimate Microsoft OneDrive & EdgeWebView2 updates.
```

- **PowerShell Event ID 4104:** Reviewed 500 recorded script blocks. No outbound staging or execution commands were found.
- **Windows BITS Logs:** All background jobs corresponded to signed Microsoft software updates.
- **The `%TEMP%\start.js` Artifact (`Worm.Nemucod.JS`):** Located an inactive 85-byte script file dated October 28 in `%TEMP%` , with no associated persistence or network calls.
- **Cross-Platform Audit (Debian Linux & Laptop):** On my desktop, I maintain a dual-boot setup with a **Debian Linux** installation, and I also work from a Linux laptop. I audited all Chromium, Brave, and Chrome profiles across both operating systems. All were **100% clean** — the infection was isolated entirely to the single default Edge profile.

---

10. Immediate Incident Response Steps

If you face a rogue extension or suspected browser compromise, follow this sequence:

Step 1: Immediate Network Isolation

1. **Disconnect from the network** (unplug Ethernet or disable Wi-Fi).
2. **Do not log into services from the compromised host** to check account status.

Step 2: Password Resets from a Known-Clean Device

Prioritize resets in this order: 1. **Primary Email & Identity Providers** (Google, Microsoft) 2. **Developer & Cloud Services** (GitHub, AWS, Cloudflare) 3. **Financial Platforms** (PayPal, online banking) 4. **Communication & Social** (Discord, LinkedIn)

Step 3: Session Revocation


```
<span class="callout-icon">🚨</span>
<span class="callout-title">Passwords Do Not Kill Active Session Cookies</span>
```

```
<p>Resetting a password does not always terminate active session tokens
```


or OAuth grants. You must explicitly select **«Sign out of all sessions / Revoke active devices»** in the security settings of each provider.

Step 4: Browser Sanitization

1. Remove all unverified extensions.
2. Clear Cookies, Local Storage, Cache, and Service Workers.
3. Inspect and clean:
   - `%LocalAppData%\Microsoft\Edge\User Data\Default\Extensions`
   - `%LocalAppData%\Microsoft\Edge\User Data\Default\Local Storage\leveldb`
   - `%LocalAppData%\Microsoft\Edge\User Data\Default\Code Cache`

Step 5: Windows Hardening

Disable the Windows Script Host for the current user to prevent accidental execution of `.vbs` and `.js` files:

```
# Disable Windows Script Host for current user
$wshPath = "HKCU:\Software\Microsoft\Windows Script Host\Settings"
if (-not (Test-Path $wshPath)) {
    New-Item -Path $wshPath -Force | Out-Null
}
Set-ItemProperty -Path $wshPath -Name "Enabled" -Value 0 -Type DWord
```

---

11. The Aftermath: When Paranoia Turns into Code (Project Sotiria)

At 3:30 AM, once the immediate triage tasks are finished —the network is cut, passwords are reset, and sessions are revoked— a lingering concern remains. Commercial endpoint tools showed green checkmarks while data was being extracted, and the browser quietly established DoH bypasses.

Looking at the Android phone sitting on the desk makes that concern concrete. The phone runs the same primary Google accounts, authenticators, and synchronized browser state. When an extension has in-process DOM access on your main workstation for 71 days, trusting standard cleanups feels insufficient.

That night led to the creation of **Sotiria (Sotiria — Endpoint Sentinel)** — named after the Greek word *Σωτηρία* (salvation and protection): an autonomous endpoint defense and remediation toolkit built for Windows (with an integrated inspection pipeline for Android), focused on neutralizing droppers, stealers, and stealthy persistence mechanisms that traditional AVs overlook.

Without revealing the entire architecture prematurely, Sotiria includes: * **Targeted Containment & YARA Rules:** Scanning staging directories for malicious signatures and isolating them in an encrypted Quarantine Vault without destructive file loss. * **System Hardening & Persistence Removal:** Cleaning rogue Scheduled Tasks and registry keys, disabling Windows Script Host, and locking down directory permissions. * **Live Process Watchdog & Host Firewall Control:** Monitoring process executions from staging paths and injecting C2 block rules into Windows Firewall in real time. * **Android Perimeter Audit via ADB:** Read-only inspection of connected Android devices to flag unauthorized Accessibility Services, sideloaded APKs, and suspicious background permissions. * **Dark-SOC Dashboard:** A standalone .NET 8 GUI with live telemetry, event logging, and audit reporting.

This tool gave me the verification I needed to close the incident safely. The full technical architecture, source code, and deployment patterns will be covered in a dedicated follow-up post.


```
<span class="callout-icon">🎯</span>
<span class="callout-title">Coming Up Next: Part 2 — Project Sotiria</span>
```

```
<p>In the next article, we move from Incident Response to <strong>Active
```


Defense: A technical breakdown of **Sotiria (Endpoint Sentinel)** — its underlying architecture, PowerShell modules, .NET 8 GUI, and how to harden Windows and Android endpoints when commercial solutions fall short. *Stay tuned!*

Conclusion

Once the initial adrenaline clears and logs confirm zero remaining malicious connections, three principles stand out:

1. **Browser Extensions Have Broad Privileges:** An extension with permission to read and change data on all websites has more direct visibility into your authenticated life than most standalone binaries. It runs in your browser context and bypasses conventional endpoint security.
2. **Blocked Packets Can Be Misleading:** The fact that an antivirus stopped 1,835 connections to one domain provided false reassurance while four other endpoints operated unchecked.
3. **Defense in Depth is Essential:** Use unique passwords with a password manager, enforce 2FA with hardware keys or authenticator apps, inspect extensions regularly, and revoke sessions at the first sign of an anomaly.

At the very least, whoever was analyzing the data intercepted from my university portal received an unwanted tutorial on the complexities of discrete mathematics in **PLH12**. That is some consolation.

Review your installed browser extensions today — and do not wait for an antivirus alert to tell you something is wrong.